ATELIER PRATIQUE GUIDÉ Programmer avec Laravel

CHAPITRE 1 – Gestion du routage

CONSIGNE IMPORTANTE

Tu dois **lire les commentaires dans le code** : ils expliquent *pourquoi* chaque ligne existe.
Ne copie pas sans comprendre : chaque commentaire correspond à une action précise de Laravel.

PARTIE 1 — ROUTAGE DE BASE

ÉTAPE 1 — Première route Laravel

📂 Fichier : routes/web.php

```
<?php
// On importe la classe Route depuis le framework Laravel
// Cette classe permet de déclarer toutes les routes de l'application
use Illuminate\Support\Facades\Route;

// Route::get signifie :
// → cette route répond uniquement à une requête HTTP GET
// → GET = affichage / lecture (navigation classique navigateur)
Route::get('/greeting', function () {

    // Le mot-clé return envoie une réponse au navigateur
```

```
// Ici, on renvoie simplement du texte
   return 'Hello World';
});
 /*Ce que fait Laravel exactement :
    1. Il attend une requête GET
    2. Il vérifie l'URL /greeting
    3. Il exécute la fonction
    4. Il renvoie le texte au navigateur*/
```

ÉTAPE 2 — Méthodes HTTP

```
// Route qui accepte une requête POST
// POST est utilisé pour envoyer des données (formulaire)
Route::post('/test-post', function () {
   // Réponse envoyée uniquement si la requête est POST
   return 'Requête POST reçue';
});
/* Important :
    • Le navigateur tape toujours du GET
    • POST / PUT / DELETE passent par des formulaires ou outils*/
```

ÉTAPE 3 — Injection de dépendance (Request)

```
// Import de la classe Request
// Elle représente la requête HTTP envoyée par le navigateur
use Illuminate\Http\Request;
```

```
Route::get('/users', function (Request $request) {

   // Laravel crée automatiquement l'objet Request
   // $request->ip() récupère l'adresse IP du visiteur
   return 'Votre adresse IP est : ' . $request->ip();
});

 /*----Tu n'as PAS écrit : new Request()
---- Laravel le fait pour toi automatiquement.*/
```

ÉTAPE 4 — Redirection

```
// Quand l'utilisateur visite /ici
// Laravel le redirige automatiquement vers /greeting
Route::redirect('/ici', '/greeting');
// Redirection permanente (SEO)
// Code HTTP 301
Route::permanentRedirect('/old', '/greeting');
```

ÉTAPE 5 — Afficher une vue sans contrôleur

resources/**views/welcome.blade.php**

```
<!-- {{ }} est une syntaxe Blade -->
<!-- Elle permet d'afficher une variable envoyée par Laravel -->
<h1>Bienvenue {{ $name }}</h1>
```

📂 routes/web.php

```
 // Route::view est un raccourci Laravel
```

```
// 1er paramètre : URL
// 2e paramètre : nom de la vue
// 3e paramètre : données envoyées à la vue
Route::view('/welcome', 'welcome', ['name' => 'Ahmed']);
```

ÉTAPE 6 — Lister les routes

Terminal :

```
php artisan route:list
```

Cette commande affiche :

- **Toutes les routes**
- **Leur URL**
- **Leur méthode**
- **Leur nom**
- **Leur middleware**

PARTIE 2 — PARAMÈTRES DE ROUTAGE

```
// {id} est un paramètre dynamique
// Il change selon l'URL tapée
Route::get('/user/{id}', function ($id) {
    // $id contient la valeur présente dans l'URL
    return 'Utilisateur numéro ' . $id;
});
```

URL /user/5 → $id = 5

Paramètre facultatif

```
// Le ? rend le paramètre optionnel
Route::get('/user/{name?}', function ($name = 'Invité') {
    // Si aucun nom n'est fourni, Invité est utilisé
    return $name;
});
```

Paramètres + Request

```
Route::get('/user/{id}', function (Request $request, $id) {
    // Règle Laravel : Request TOUJOURS en premier
    return 'User ' . $id;
});
```

PARTIE 3 — ROUTES NOMMÉES

```
Route::get('/user/profile', function () {
    return 'Profil utilisateur';
})
// Le nom permet de référencer la route sans écrire l'URL
->name('profile');
// Génération d'URL
route('profile');
// Redirection vers une route nommée
redirect()->route('profile');
```

PARTIE 4 — GROUPE DE ROUTES

```
Route::group([
    // Préfixe commun pour toutes les routes du groupe
    'prefix' => 'admin'
], function () {
    // URL finale : /admin
    Route::get('/', function () {
        return 'Admin dashboard';
    });
    // URL finale : /admin/logs
    Route::get('/logs', function () {
        return 'Logs admin';
    });
});
```

PARTIE 5 — LIAISON DE MODÈLES

```
// Import du modèle User
use App\Models\User;
Route::get('/users/{user}', function (User $user) {
    // Laravel exécute automatiquement :
    // User::find(id)
    return $user->email;
});
/* -----Si l'utilisateur n'existe pas ➜ 404 automatique */
```

PARTIE 6 — ROUTE DE REPLI

```
Route::fallback(function () {
   // Cette route s'exécute si aucune autre ne correspond
   return 'Page non trouvée';
});
```

PARTIE 7 — SPOOFING DES FORMULAIRES

```
<!-- Début du formulaire HTML -->
<form action="/example" method="POST">
   <!—1/ CSRF : Cross-Site Request Forgery
        Pourquoi ?
        - CSRF est une attaque où un site malveillant pourrait envoyer des requêtes à ton site en utilisant l'identité d'un utilisateur connecté.
        - Laravel protège automatiquement les formulaires grâce à un "token CSRF".
        Comment ça marche ?
        - @csrf génère un champ caché <input type="hidden"> contenant un token unique pour la session utilisateur.
        - Laravel vérifie ce token à la réception du formulaire. Si le token est absent ou incorrect, la requête est rejetée.
        Résultat : ton formulaire est sécurisé automatiquement.
   -->
   @csrf

   <!—2/ Spoofing de méthode HTTP
        Problème :
```

- Les formulaires HTML ne supportent que GET et POST.

- On veut parfois faire des PUT, PATCH ou DELETE (modifier ou supprimer des données).

Solution Laravel :

- @method('PUT') génère un champ caché <input name="_method" value="PUT">

*- Laravel lit ce champ et **transforme la requête POST en PUT** automatiquement.*

Résultat :

- Tu peux utiliser PUT / DELETE via un formulaire classique.

- Exemple : modifier un utilisateur via PUT.

-->

```
@method('PUT')
<!—3/ Bouton pour envoyer le formulaire -->
<button type="submit">Modifier</button>
</form>
```

*<!—***----- Explications supplémentaires**

@csrf

Obligatoire dans tous les formulaires POST, PUT, DELETE.

Sans ça, Laravel refusera la requête → erreur 419 (CSRF token mismatch).

@method('PUT')

“Spoofing” = usurpation de la méthode HTTP.

HTML ne sait pas envoyer PUT / DELETE.

Laravel convertit la requête POST en PUT / DELETE grâce à ce champ caché.

<button type="submit">

Bouton classique qui envoie le formulaire.

Si tu oublies, le formulaire ne s'enverra jamais. -->

PARTIE 8 — ROUTE COURANTE

```
use Illuminate\Support\Facades\Route;
Route::get('/test', function () {
   // Retourne le nom de la route actuelle
   return Route::currentRouteName();
});
```

PARTIE 9 — CORS & CACHE DES ROUTES

📂 Configuration CORS : config/cors.php

```
# Mise en cache des routes (production)
php artisan route:cache
# Suppression du cache
php artisan route:clear
```

-------NB----- **Ne jamais utiliser route:cache en développement**